FHIR SME Program: Large POC Architecture & Environment Setup Verification Log

Comprehensive Engineering Log with Core SME Learning Vectors | CMS-0057 Interoperability Architecture

Engineer / Architect: Ashish Singh	**Target Standard:** CMS-0057-F & HL7 FHIR R4 (4.0.1)	**Execution Date:** 12 October 2026
Root Repository: /Users/ashishsingh/Projects/CMS-Interop	**GCP Project ID:** project-1c617076-69b8-413f-94d	**GCP Store:** us-central1 (fhir-lab / r4-store)
Firely Server: Docker latest (Port 4080 / arm64)	**License Status:** Evaluation (Valid to 2026-11-10)	**Epic User:** Thetallman (fhir.epic.com)

1. SME Learning & POC Mastery Matrix (Why, What, How & Lessons Learned)

This section documents the architectural rationale, domain concepts learned, technical verification methods, and hard-earned engineering lessons across all environment initialization steps.

Step #	Component	Why I Did It (Architectural Rationale)	What I Learned (Core Healthcare/FHIR Concept)	How I Learned (Hands-on Technical Method)	Lesson Learned & Pitfall Avoided (Production Insight)	Success	Complete
0.1	**Host Prerequisites**	Establish clean runtime foundation for Docker containers, cloud SDKs, JVM validator CLI, and Python scripting.	Interoperability pipelines require heterogeneous tooling: Docker for local conformance, gcloud CLI for cloud stores, Java 17 for HL7 validator CLI, Python for CDS Hooks.	Audited system binaries via terminal: docker --version, gcloud config, java OpenJDK 26, python3, jq. Initialized Docker daemon on macOS arm64.	Docker daemon on macOS requires explicit startup before container commands execute. Verifying all prerequisites early prevents cryptic failures during pipeline runs.	**Y**	**Y**
3.1	**Workspace & Variables**	Isolate configuration in env.sh and credentials in env.local.sh to adhere to 12-factor design and HIPAA credential isolation.	Standardized FHIR request formatting. Custom helper functions fire() and gcp() enforce mandatory 'application/fhir+json' MIME types and handle Google IAM OAuth2 tokens.	Structured env.sh with PROJECT, REGION, FIRELY, GCP, EPIC; sourced private tokens via env.local.sh (.gitignored). Verified variable inheritance and function execution.	Omitting 'Content-Type: application/fhir+json' causes FHIR servers to reject payloads with 415 Unsupported Media Type or return unformatted HTML errors.	**Y**	**Y**
3.2	**Firely Conformance Gate**	Deploy an independent, standards-strict FHIR R4 server to act as the payer-side conformance validator, CDS Hooks server, and bulk data exporter.	Firely (Vonk) uses modular plugins. Evaluation license controls enabled plugins (CDS Hooks, SMART, Bulk Export). Conformance resources load into memory/SQLite on boot.	Mounted firelyserver-license.json (valid to 2026-11-10) into Docker container on port 4080. Verified log: 'Conformance resource loading finished. Server ready to receive requests.'	Container exits immediately if the license file path or mount is incorrect. Firely serves as the strict conformance gate, preventing malformed data from reaching production stores.	**Y**	**Y**
3.3	**GCP FHIR Store Setup**	Provision a durable, enterprise-scale R4 FHIR store (Cloud Healthcare API) with native IAM, audit logging, and direct streaming export to BigQuery.	Differences between an in-memory test server and enterprise cloud store. Enabling '--enable-update-create' allows client-assigned ID PUTs (PUT /Patient/{id}) for migration reconciliation.	Verified Healthcare API, inspected dataset 'fhir-lab' and store 'r4-store' in us-central1. Tested metadata endpoint using Google OAuth2 Bearer token: returns FHIR 4.0.1.	GCP enforces referential integrity by default; orphaned references cause HTTP 400. Update-create must be explicitly enabled to preserve original IDs across multi-platform hops.	**Y**	**Y**
3.4	**Epic Sandbox & PKCE Auth**	Connect to a real-world EHR (Epic) to implement both CMS-0057 Patient Access API (PKCE public client) and backend automated extraction (signed JWT).	Duality of healthcare auth: Patient apps use OAuth2 Authorization Code with PKCE (S256) because browsers cannot secure secrets. Backend apps use private_key_jwt with RS384.	Registered 'CMS-Interop Patient App' (eb8c8524...) and 'CMS-Interop Backend App' (a3e7d395...). Executed live PKCE browser handshake and logged into MyChart as Camila Lopez.	Navigating directly to /oauth2/authorize without query parameters causes an OAuth2 error. SMART on FHIR requires exact query parameters (client_id, redirect_uri, scope, aud, code_challenge).	**Y**	**Y**
3.5	**Tri-Platform Smoke Test**	Execute automated tri-platform handshake to verify that all three tiers (EHR source, conformance gate, enterprise store) speak the identical FHIR version.	Every compliant FHIR server publishes its CapabilityStatement at /metadata. Querying .fhirVersion confirms API contract compatibility across disparate vendors.	Executed concurrent metadata calls across Firely (localhost:4080), GCP Healthcare API, and Epic on FHIR. All 3 returned HTTP 200 with FHIR version '4.0.1'.	Version mismatches (e.g. STU3 vs R4) cause silent schema corruption (e.g. MedicationOrder vs MedicationRequest). Verifying 4.0.1 across all 3 tiers guarantees pipeline interoperability.	**Y**	**Y**

Step #	Component	Why I Did It (Architectural Rationale)	What I Learned (Core Healthcare/FHIR Concept)	How I Learned (Hands-on Technical Method)	Lesson Learned & Pitfall Avoided (Production Insight)	Success	Complete
3.6	**Python Clinical Venv**	Create dedicated Python virtual environment (.venv) containing modern clinical libraries (FastAPI, Uvicorn, PyJWT, Cryptography, Requests).	How modern FHIR services operate. FastAPI provides asynchronous CDS Hooks discovery; PyJWT and Cryptography verify signed JWT assertions against JWKS without third-party libraries.	Provisioned .venv in CMS-Interop root repo; installed clinical packages. Tested load_examples.py --dry-run with 0 errors across 65 resources.	Never use global Python packages for clinical pipelines. Isolated virtual environments guarantee deterministic cryptography and prevent dependency conflicts during certification testing.	**Y**	**Y**
3.7	**Single-Resource Ingestion POC**	Avoid blind bulk loading. Validate, inspect, and ingest individual foundation and clinical resources from CSV to Firely before mass population.	FHIR topological dependency order: Foundation resources (Organization, Location, Practitioner) must exist before Clinical resources (Patient, Observation). Profiles enforce US Core extensions.	Developed create_single_resource.py. Validated Seq 1 (Organization/org-payer) and Seq 9 (Patient/asha-001 with US Core race/ethnicity/birthsex) using Firely $validate, PUT, and GET.	A successful HTTP 200 PUT does not guarantee profile conformance. Running $validate explicitly catches missing extensions, invalid OIDs, and cardinalities before database commit.	**Y**	**Y**
3.8	**Dual-Store Cohort Ingestion**	Populate full 65-resource end-to-end clinical & payer test cohort across Firely and GCP stores; diagnose multi-server conformance discrepancies.	Different FHIR servers apply conformance rules differently: Firely strictly validates QuestionnaireResponse against Questionnaire definitions; GCP enforces FHIRPath constraint ppc-1 on Consent.	Executed load_examples.py --target both --verify. Diagnosed HTTP 400 on Seq 50 (contained Questionnaire fix) and Seq 60 (ppc-1 policy/policyRule fix). Re-ran and verified 65/65 readable on both servers.	Production FHIR data rarely passes multi-vendor ingestion on first attempt. Always inspect OperationOutcome diagnostics, resolve contained definitions, and satisfy core invariants before migration.	**Y**	**Y**

2. Technical Execution & Operational Verification Log

This section documents the specific CLI commands, API calls, observed outputs, and concrete success criteria for each task.

Step #	Task / Component	Commands / Scripts Run	Observed Results & Technical Proof	Success Criteria	Status

0.1	**Prerequisites Audit**	`docker --version`
		`gcloud config list`
		`java -version`
		`python3 --version`
		`jq --version`
			Docker 28.3.3; gcloud SDK 577.0.0 (account: monikamalik281112@gmail.com, project: project-1c617076-69b8-413f-94d); OpenJDK 26; Python 3.14.4; jq 1.7.1.	All core binaries operable on macOS arm64 host architecture.	**COMPLETE (Y)**

3.1	**Workspace & Variables**	`cat /Users/ashishsingh/Projects/CMS-Interop/env.sh`
		`source env.sh`
		`type fire gcp`
			PROJECT, REGION, FIRELY, GCP, EPIC URLs exported. Sourcing env.local.sh enabled. Helper functions fire() and gcp() active with application/fhir+json.	Clean shell variable export; helper functions functional across terminals.	**COMPLETE (Y)**

3.2	**Firely Container Deploy**	`docker run -d --name firely -p 4080:4080 -v ...:firelyserver-license.json firely/server:latest` `docker logs firely`	Container 0cd831c65e healthy. Conformance resources loaded from specification_Fhir4_0.zip. Log: 'Server is ready to receive requests.'	Container stays up, valid evaluation license (Ashish.Singh@abacusinsights.com, exp 2026-11-10), port 4080 bound.	**COMPLETE (Y)**

3.3	**GCP Store Validation**	`gcloud healthcare datasets list`
		`gcloud healthcare fhir-stores list`
		`gcp $GCP/metadata | jq -r .fhirVersion`
			Dataset: fhir-lab (us-central1). Store: r4-store (FHIR R4). update-create: True. Response: 4.0.1.	Managed store active, IAM OAuth2 Bearer token authentication verified, update-create enabled.	**COMPLETE (Y)**

3.4	**Epic Sandbox & MyChart**	`Created apps on fhir.epic.com.`
		`Saved client IDs in env.local.sh.`
		`Executed PKCE browser handshake.`
			Patient App (PKCE): eb8c8524-e473-4c78-87a8-d15c4ad6dd2b.
			Backend App (JWT): a3e7d395-2d50-4491-9e68-bd35c533623b.
			Browser successfully authenticated into MyChart as Camila Lopez.
				Both non-prod client IDs recorded in env.local.sh; interactive MyChart login verified.	**COMPLETE (Y)**

3.5	**Smoke Test Handshake**	`curl -s $FIRELY/metadata | jq -r .fhirVersion`
		`gcp $GCP/metadata | jq -r .fhirVersion`
		`curl -s -H 'Accept: application/fhir+json' $EPIC/metadata | jq -r .fhirVersion`
			Firely: 4.0.1
			GCP: 4.0.1
			Epic: 4.0.1
				All three separate FHIR servers return HTTP 200 and report identical FHIR version 4.0.1.	**COMPLETE (Y)**

3.6	**Python Clinical Venv**	`python3 -m venv .venv` `pip install fastapi uvicorn httpx pyjwt cryptography requests`	Installed anyio, certifi, cffi, cryptography 50.0.2, fastapi 0.143.0, httpx 0.28.1, pyjwt 2.15.1, requests 2.34.2, uvicorn 0.54.0. Tested imports with 0 errors.	Virtual environment isolated in root repo; ready for CRD server (Lab 4.1) and SMART client (Lab 5.2).	**COMPLETE (Y)**
3.7	**Single Resource Ingestion**	`python create_single_resource.py --seq 1` `python create_single_resource.py --path Patient/asha-001`	Seq 1: Organization/org-payer ($validate: 200, PUT: 200, VersionId: df4fb8bd...). Seq 9: Patient/asha-001 with US Core race/ethnicity/birthsex ($validate: 200, PUT: 200, VersionId: 900c19ea...).	Manual/single-step ingestion verified on Firely Server before batch population; profile conformance pre-validated.	**COMPLETE (Y)**

3.8	**Dual-Store Cohort Ingestion**	`python load_examples.py --target both --verify`
		`# Quarantine diagnosis & resolution:`
		`POST /QuestionnaireResponse/$validate`
		`POST /Consent/$validate`
			Initial load: Firely 63/65 (400 on Seq 50, 60), GCP 64/65 (400 on Seq 60).
			Diagnosed & resolved: Contained Questionnaire (Seq 50) + ppc-1 policy/policyRule (Seq 60).
			Re-run: Firely 65/65 readable, GCP 65/65 readable.
				All 65 clinical & administrative resources loaded and verified via HTTP GET on both Firely and GCP stores.	**COMPLETE (Y)**

3. Deep-Dive Ingestion Walkthrough & Technical Proof

Step-by-Step Single-Resource Ingestion POC (Demonstrating Step 3.7):
Rather than blindly running batch scripts, an SME verifies how raw tabular/CSV data translates into FHIR R4 resources, validates the payload against the server's conformance engine, and checks database commits.

1. Foundation Payer Organization (Seq 1: Organization/org-payer):
• **Payload Structure:** Contains standard identifier (http://example.org/ids/org), type coded with HL7 v2-0203 system (pay), and name: "Lab Health Plan".
• **Conformance Gate ($validate):** Called POST /Organization/$validate. Firely returned HTTP 200 with 0 issues.
• **Idempotent Write (PUT):** Called PUT /Organization/org-payer. Firely assigned version ID df4fb8bd-3684-4a6f-97ab-d9cdbc1c912a.
• **Read-back Verification:** GET /Organization/org-payer returned active status and verified the stored record.

2. Clinical Patient with US Core Profile (Seq 9: Patient/asha-001):
• **Payload Structure:** Conforms to http://hl7.org/fhir/us/core/StructureDefinition/us-core-patient. Carries 3 mandatory US Core extensions:
  - Race: http://hl7.org/fhir/us/core/StructureDefinition/us-core-race (OMB code 2106-3: White)
  - Ethnicity: http://hl7.org/fhir/us/core/StructureDefinition/us-core-ethnicity (OMB code 2186-5: Not Hispanic or Latino)
  - Birthsex: http://hl7.org/fhir/us/core/StructureDefinition/us-core-birthsex (Value: F)
• **References:** Linked to Practitioner/prac-001 (PCP) and Organization/org-clinic (managing organization).
• **Execution:** Validated on Firely (HTTP 200), committed via PUT (Version ID: 900c19ea-7288-4ea9-86a8-7accd8325321).

3. Dual-Store Full Cohort Ingestion & Quarantine Resolution (Step 3.8):
• **Cohort Scale:** 65 resources covering 4 patient journeys (Asha Sharma - T2DM/HTN, Ravi Patel - TKA/knee, Meera Shah - Asthma/Allergy, John Doe - MRI Prior Auth).
• **Discrepancy 1 (Seq 50 QuestionnaireResponse/qr-john-mri):** Firely rejected with 400 (QuestionnaireResponseValidator: Failed to resolve Questionnaire reference). GCP accepted it. *Root cause:* Firely strictly validates QuestionnaireResponse against Questionnaire definitions. Resolved by embedding the Questionnaire as a contained resource (#q-mri-brain-dtr), satisfying FHIR constraint dom-3. Firely $validate returned HTTP 200 (0 issues).
• **Discrepancy 2 (Seq 60 Consent/consent-asha-sharing):** GCP rejected with 400 (fhirpath-constraint-violation-Consent: "policy.exists() or policyRule.exists()"). *Root cause:* FHIR R4 invariant ppc-1 mandates policy or policyRule even when provision is present. Resolved by adding policy URI and policyRule coding (OPTIN). Both servers accepted with HTTP 200/201.
• **Final Verification:** load_examples.py --target both --verify confirmed **65/65 readable on Firely** and **65/65 readable on GCP** (100% parity).

Three-Tier POC Operating Pipeline Architecture:

Tier / Role	Platform & Endpoint	Authentication & Protocols	Core Role in CMS-0057 Program
1. EHR Source	Epic on FHIR Sandbox https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4	OAuth2 + SMART on FHIR • Patient: PKCE S256 (MyChart) • Backend: Signed JWT (RS384)	Simulates source provider EHR. Supplies synthetic patients (Camila Lopez) for USCDI and EHI extraction.
2. Conformance Gate	Firely Server (vonk) http://localhost:4080	Evaluation License • CDS Hooks Host • FHIR $validate Engine	Payer-side conformance filter. Validates incoming data against US Core and Da Vinci IGs, hosts CRD services.
3. System of Record	GCP Cloud Healthcare API https://healthcare.googleapis.com/.../fhir-lab/fhirStores/r4-store/fhir	Google IAM OAuth2 • Bearer Token via gcloud • Update-Create Enabled	Durable multi-region FHIR repository. Houses 5-year longitudinal records and streams data to BigQuery for analytics.

POC Environment & Labs Sign-off: 100% COMPLETE & VERIFIED	**Milestone Complete: Setup & Module 1 Labs 1.1-1.3 (Saved in labs/Module1_FHIR_Fundamentals_Complete.docx)**